Mathematical Foundation of Recurrent Neural Networks: A Detailed Mathematical Analysis

Achuth

June 9, 2025

Contents

1 Introduction to Sequential Data Processing

Before diving into the mathematical formulation of Recurrent Neural Networks, we must establish the fundamental problem they solve. Traditional feedforward neural networks process inputs independently, lacking the ability to maintain information across time steps. This limitation becomes critical when dealing with sequential data where context and temporal dependencies are essential.

Consider a sequence of inputs $\mathbf{x} = (x_1, x_2, \ldots, x_T)$ where T represents the sequence length. Each element x_t at time step t depends not only on its own properties but also on the context established by previous elements $x_1, x_2, \ldots, x_{t-1}$. Recurrent Neural Networks address this challenge by introducing a memory mechanism that allows information to persist across time steps.

2 Basic RNN Mathematical Formulation

2.1 Core Equations

The mathematical foundation of a basic RNN rests on three fundamental equations that define how information flows through the network. Let us examine each equation in detail.

The hidden state at time step t is computed as:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h) \tag{1}$$

This equation represents the core recurrent computation. The hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$ at time t is computed by combining three components: the previous hidden state $\mathbf{h}_{t-1}$ transformed by weight matrix $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$, the current input $\mathbf{x}_t \in \mathbb{R}^{d_x}$ transformed by weight matrix $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times d_x}$, and a bias vector $\mathbf{b}_h \in \mathbb{R}^{d_h}$.

The output at time step t is given by:

$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y \tag{2}$$

Here, $\mathbf{y}_t \in \mathbb{R}^{d_y}$ represents the output at time t, computed by applying an affine transformation to the hidden state using weight matrix $\mathbf{W}_{hy} \in \mathbb{R}^{d_y \times d_h}$ and bias vector $\mathbf{b}_y \in \mathbb{R}^{d_y}$.

For classification tasks, we typically apply a softmax function to obtain probability distributions:

$$P(y_t = k) = \frac{\exp(y_{t,k})}{\sum_{j=1}^{K} \exp(y_{t,j})} \tag{3}$$

where K is the number of classes and $y_{t,k}$ is the k-th component of $\mathbf{y}_t$.

2.2 Initial Conditions

The recurrent computation requires an initial hidden state $\mathbf{h}_0$. This is typically initialized as:

$$\mathbf{h}_0 = \mathbf{0} \quad \text{or} \quad \mathbf{h}_0 \sim \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I}) \tag{4}$$

The choice of initialization can impact the network's ability to capture dependencies from the beginning of sequences.

2.3 Unfolding Through Time

To understand how RNNs process entire sequences, we can unfold the recurrent computation through time. For a sequence of length T, the complete computation becomes:

$$\mathbf{h}_1 = \tanh(\mathbf{W}_{hh}\mathbf{h}_0 + \mathbf{W}_{xh}\mathbf{x}_1 + \mathbf{b}_h) \tag{5}$$
$$\mathbf{h}_2 = \tanh(\mathbf{W}_{hh}\mathbf{h}_1 + \mathbf{W}_{xh}\mathbf{x}_2 + \mathbf{b}_h) \tag{6}$$
$$\vdots \tag{7}$$
$$\mathbf{h}_T = \tanh(\mathbf{W}_{hh}\mathbf{h}_{T-1} + \mathbf{W}_{xh}\mathbf{x}_T + \mathbf{b}_h) \tag{8}$$

This unfolding reveals that the hidden state at any time t implicitly contains information from all previous inputs $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_t$.

3 Backpropagation Through Time (BPTT)

Training RNNs requires a specialized form of backpropagation called Backpropagation Through Time. This algorithm extends standard backpropagation to handle the temporal dependencies in recurrent networks.

3.1 Loss Function

For a sequence of length T, the total loss is typically the sum of losses at individual time steps:

$$L = \sum_{t=1}^{T} L_t(\mathbf{y}_t, \hat{\mathbf{y}}_t) \tag{9}$$

where L_t is the loss at time step t, $\mathbf{y}_t$ is the predicted output, and $\hat{\mathbf{y}}_t$ is the target output.

For cross-entropy loss, commonly used in classification tasks:

$$L_t = -\sum_{k=1}^{K} \hat{y}_{t,k} \log(y_{t,k}) \tag{10}$$

3.2 Gradient Computation

The gradient computation in BPTT involves both spatial gradients (across network layers) and temporal gradients (across time steps). Let us derive the gradients for each parameter.

3.2.1 Output Layer Gradients

The gradient with respect to the output weight matrix $\mathbf{W}_{hy}$ is:

$$\frac{\partial L}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial \mathbf{y}_t} \mathbf{h}_t^T \tag{11}$$

The gradient with respect to the output bias $\mathbf{b}_y$ is:

$$\frac{\partial L}{\partial \mathbf{b}_y} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial \mathbf{y}_t} \tag{12}$$

3.2.2 Hidden State Gradients

The gradient with respect to the hidden state at time t requires careful consideration of how $\mathbf{h}_t$ affects both the current output and future hidden states:

$$\frac{\partial L}{\partial \mathbf{h}_t} = \frac{\partial L_t}{\partial \mathbf{y}_t}\frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t} + \frac{\partial L}{\partial \mathbf{h}_{t+1}}\frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t} \tag{13}$$

This can be written more explicitly as:

$$\frac{\partial L}{\partial \mathbf{h}_t} = \mathbf{W}_{hy}^T\frac{\partial L_t}{\partial \mathbf{y}_t} + \mathbf{W}_{hh}^T\text{diag}(1-\tanh^2(\mathbf{a}_{t+1}))\frac{\partial L}{\partial \mathbf{h}_{t+1}} \tag{14}$$

where $\mathbf{a}_{t+1} = \mathbf{W}_{hh}\mathbf{h}_t + \mathbf{W}_{xh}\mathbf{x}_{t+1} + \mathbf{b}_h$ is the pre-activation at time $t+1$.

3.2.3 Weight Matrix Gradients

The gradient with respect to the recurrent weight matrix $\mathbf{W}_{hh}$ accumulates contributions from all time steps:

$$\frac{\partial L}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T}\frac{\partial L}{\partial \mathbf{h}_t}\text{diag}(1-\tanh^2(\mathbf{a}_t))\mathbf{h}_{t-1}^T \tag{15}$$

Similarly, the gradient with respect to the input weight matrix $\mathbf{W}_{xh}$ is:

$$\frac{\partial L}{\partial \mathbf{W}_{xh}} = \sum_{t=1}^{T}\frac{\partial L}{\partial \mathbf{h}_t}\text{diag}(1-\tanh^2(\mathbf{a}_t))\mathbf{x}_t^T \tag{16}$$

The gradient with respect to the hidden bias $\mathbf{b}_h$ is:

$$\frac{\partial L}{\partial \mathbf{b}_h} = \sum_{t=1}^{T}\frac{\partial L}{\partial \mathbf{h}_t}\text{diag}(1-\tanh^2(\mathbf{a}_t)) \tag{17}$$

3.3 The Vanishing Gradient Problem

The vanishing gradient problem in RNNs can be understood mathematically by examining how gradients propagate through time. Consider the gradient of the loss with respect to $\mathbf{h}_k$ for $k < t$:

$$\frac{\partial L}{\partial \mathbf{h}_k} = \frac{\partial L}{\partial \mathbf{h}_t}\prod_{i=k+1}^{t}\frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}} \tag{18}$$

Each term in the product is:

$$\frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}} = \mathbf{W}_{hh}^T\text{diag}(1-\tanh^2(\mathbf{a}_i)) \tag{19}$$

Since $|\tanh'(x)| \leq 1$, and typically much smaller, the product tends to vanish exponentially as $(t-k)$ increases. Specifically, if the largest eigenvalue of $\mathbf{W}_{hh}$ is λ_{max}, then:

$$\left\|\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}\right\| \leq (\lambda_{max})^{t-k}\prod_{i=k+1}^{t}\|\text{diag}(1-\tanh^2(\mathbf{a}_i))\| \tag{20}$$

When $\lambda_{max} < 1$, this bound decreases exponentially with $(t-k)$, leading to vanishing gradients.

4 Long Short-Term Memory (LSTM) Networks

LSTM networks address the vanishing gradient problem through a more sophisticated memory mechanism involving gates that control information flow.

4.1 LSTM Cell State Equations

An LSTM cell maintains two types of state: the cell state $\mathbf{C}_t$ and the hidden state $\mathbf{h}_t$. The cell state acts as a highway for information flow, while the hidden state serves as the output of the cell.

The forget gate determines what information to discard from the cell state:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \tag{21}$$

The input gate decides what new information to store:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \tag{22}$$

New candidate values are created:

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_C \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C) \tag{23}$$

The cell state is updated by combining old and new information:

$$\mathbf{C}_t = \mathbf{f}_t * \mathbf{C}_{t-1} + \mathbf{i}_t * \tilde{\mathbf{C}}_t \tag{24}$$

The output gate controls what parts of the cell state to output:

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \tag{25}$$

Finally, the hidden state is computed:

$$\mathbf{h}_t = \mathbf{o}_t * \tanh(\mathbf{C}_t) \tag{26}$$

Here, σ denotes the sigmoid function, $*$ denotes element-wise multiplication, and $[\mathbf{h}_{t-1}, \mathbf{x}_t]$ represents concatenation of vectors.

4.2 LSTM Gradient Flow Analysis

The key insight of LSTMs is that gradients can flow through the cell state with minimal modification. The gradient of the loss with respect to the cell state $\mathbf{C}_t$ is:

$$\frac{\partial L}{\partial \mathbf{C}_t} = \frac{\partial L}{\partial \mathbf{h}_t}\frac{\partial \mathbf{h}_t}{\partial \mathbf{C}_t} + \frac{\partial L}{\partial \mathbf{C}_{t+1}}\frac{\partial \mathbf{C}_{t+1}}{\partial \mathbf{C}_t} \tag{27}$$

Since $\frac{\partial \mathbf{C}_{t+1}}{\partial \mathbf{C}_t} = \mathbf{f}_{t+1}$, the gradient flow through the cell state is:

$$\frac{\partial L}{\partial \mathbf{C}_t} = \frac{\partial L}{\partial \mathbf{h}_t}\mathbf{o}_t * (1 - \tanh^2(\mathbf{C}_t)) + \frac{\partial L}{\partial \mathbf{C}_{t+1}}\mathbf{f}_{t+1} \tag{28}$$

Unlike basic RNNs, this gradient flow is controlled by the forget gate $\mathbf{f}_{t+1}$, which can learn to preserve gradients when long-term dependencies are important.

5 Gated Recurrent Unit (GRU)

GRUs simplify the LSTM architecture while maintaining much of its effectiveness. The GRU combines the forget and input gates into a single update gate and merges the cell and hidden states.

5.1 GRU Mathematical Formulation

The update gate determines how much of the past information to keep:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z) \tag{29}$$

The reset gate decides how much of the past information to forget:

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r) \tag{30}$$

The candidate hidden state is computed:

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \cdot [\mathbf{r}_t * \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h) \tag{31}$$

The final hidden state combines old and new information:

$$\mathbf{h}_t = (1 - \mathbf{z}_t) * \mathbf{h}_{t-1} + \mathbf{z}_t * \tilde{\mathbf{h}}_t \tag{32}$$

5.2 GRU Gradient Analysis

The gradient flow in GRUs is similar to LSTMs but simpler. The gradient with respect to the hidden state at time $t-1$ is:

$$\frac{\partial L}{\partial \mathbf{h}_{t-1}} = \frac{\partial L}{\partial \mathbf{h}_t} \left[(1 - \mathbf{z}_t) + \mathbf{z}_t \frac{\partial \tilde{\mathbf{h}}_t}{\partial \mathbf{h}_{t-1}} \right] + \text{gate gradients} \tag{33}$$

The update gate $\mathbf{z}_t$ controls the gradient flow, allowing the network to learn when to preserve or update information.

6 Bidirectional RNNs

Bidirectional RNNs process sequences in both forward and backward directions to capture both past and future context.

6.1 Mathematical Formulation

For a bidirectional RNN, we maintain two hidden states:

$$\overrightarrow{\mathbf{h}}_t = \tanh(\mathbf{W}_{\overrightarrow{hh}} \overrightarrow{\mathbf{h}}_{t-1} + \mathbf{W}_{\overrightarrow{xh}} \mathbf{x}_t + \mathbf{b}_{\overrightarrow{h}}) \tag{34}$$

$$\overleftarrow{\mathbf{h}}_t = \tanh(\mathbf{W}_{\overleftarrow{hh}} \overleftarrow{\mathbf{h}}_{t+1} + \mathbf{W}_{\overleftarrow{xh}} \mathbf{x}_t + \mathbf{b}_{\overleftarrow{h}}) \tag{35}$$

The final hidden representation combines both directions:

$$\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] \tag{36}$$

where $[;]$ denotes concatenation.

7 Attention Mechanisms in RNNs

Attention mechanisms allow RNNs to focus on different parts of the input sequence when producing outputs.

7.1 Basic Attention Mathematics

Given encoder hidden states $\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_T$ and decoder hidden state $\mathbf{s}_t$, attention weights are computed as:

$$e_{t,i} = \mathbf{v}_a^T \tanh(\mathbf{W}_a \mathbf{s}_t + \mathbf{U}_a \mathbf{h}_i + \mathbf{b}_a) \tag{37}$$

The attention weights are normalized using softmax:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T} \exp(e_{t,j})} \tag{38}$$

The context vector is a weighted sum of encoder hidden states:

$$\mathbf{c}_t = \sum_{i=1}^{T} \alpha_{t,i} \mathbf{h}_i \tag{39}$$

7.2 Attention Gradient Computation

The gradient with respect to encoder hidden states through attention is:

$$\frac{\partial L}{\partial \mathbf{h}_i} = \sum_{t=1}^{T'} \frac{\partial L}{\partial \mathbf{c}_t} \alpha_{t,i} + \sum_{t=1}^{T'} \frac{\partial L}{\partial \mathbf{c}_t} \sum_{j=1}^{T} \mathbf{h}_j \frac{\partial \alpha_{t,j}}{\partial \mathbf{h}_i} \tag{40}$$

where T' is the length of the decoder sequence.

8 Training Optimization Techniques

8.1 Gradient Clipping

To prevent exploding gradients, we clip gradients when their norm exceeds a threshold:

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g} & \text{if } \|\mathbf{g}\| \leq \theta \\ \frac{\theta}{\|\mathbf{g}\|} \mathbf{g} & \text{if } \|\mathbf{g}\| > \theta \end{cases} \tag{41}$$

where θ is the clipping threshold and $\mathbf{g}$ represents the gradient vector.

8.2 Truncated Backpropagation Through Time

For very long sequences, we can truncate the gradient computation to a fixed number of steps k:

$$\frac{\partial L_t}{\partial \mathbf{h}_{t-j}} = \begin{cases} \text{computed normally} & \text{if } j \leq k \\ 0 & \text{if } j > k \end{cases} \tag{42}$$

This reduces computational complexity from $O(T^2)$ to $O(kT)$ while maintaining reasonable gradient estimates.

9 Loss Functions for Sequential Tasks

9.1 Sequence-to-Sequence Loss

For sequence-to-sequence tasks, the loss is typically:

$$L = -\sum_{t=1}^{T} \log P(\mathbf{y}_t | \mathbf{y}_{<t}, \mathbf{x}) \tag{43}$$

where $\mathbf{y}_{<t}$ represents all previous target tokens.

9.2 Connectionist Temporal Classification (CTC)

For sequence labeling without explicit alignment, CTC loss is defined as:

$$L_{CTC} = -\log \sum_{\mathbf{a} \in \mathcal{A}(\mathbf{y})} \prod_{t=1}^{T} P(a_t | \mathbf{x}) \tag{44}$$

where $\mathcal{A}(\mathbf{y})$ is the set of all possible alignments that produce the target sequence $\mathbf{y}$.

10 Regularization in RNNs

10.1 Dropout in Recurrent Networks

Dropout in RNNs requires special consideration. We typically apply dropout to inputs and outputs but not to recurrent connections:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}(\mathbf{x}_t \odot \mathbf{m}_t) + \mathbf{b}_h) \tag{45}$$

where $\mathbf{m}_t$ is a binary mask with elements drawn from Bernoulli distribution.

10.2 Weight Decay

L2 regularization is applied to all weight matrices:

$$L_{reg} = \frac{\lambda}{2} \left(\|\mathbf{W}_{hh}\|_F^2 + \|\mathbf{W}_{xh}\|_F^2 + \|\mathbf{W}_{hy}\|_F^2 \right) \tag{46}$$

where λ is the regularization coefficient and $\|\cdot\|_F$ denotes the Frobenius norm.

11 Computational Complexity Analysis

11.1 Time Complexity

For a sequence of length T with hidden dimension d_h and input dimension d_x:

- Forward pass: $O(T \cdot d_h^2 + T \cdot d_h \cdot d_x)$
- Backward pass: $O(T \cdot d_h^2 + T \cdot d_h \cdot d_x)$
- Total training complexity: $O(T \cdot d_h^2 + T \cdot d_h \cdot d_x)$

11.2 Space Complexity

The space complexity includes:

- Hidden states storage: $O(T \cdot d_h)$
- Gradient storage: $O(T \cdot d_h)$
- Parameter storage: $O(d_h^2 + d_h \cdot d_x + d_h \cdot d_y)$

Total space complexity: $O(T \cdot d_h + d_h^2 + d_h \cdot d_x + d_h \cdot d_y)$

12 Conclusion

This mathematical analysis provides a comprehensive foundation for understanding Recurrent Neural Networks. The key insights include:

1. The recurrent connection enables information persistence through hidden states
2. Backpropagation through time extends standard backpropagation to temporal dependencies
3. The vanishing gradient problem limits basic RNNs' ability to capture long-term dependencies
4. LSTM and GRU architectures address gradient flow issues through gating mechanisms
5. Attention mechanisms provide direct access to all encoder states, bypassing recurrent bottlenecks

Understanding these mathematical foundations enables practitioners to make informed decisions about architecture choices, training procedures, and optimization strategies for sequential learning tasks.

The evolution from basic RNNs to more sophisticated architectures demonstrates how mathematical insights into gradient flow and information processing can lead to significant practical improvements in deep learning systems.